SC4055: Introduction to Quantum Computing

Project Title: Training Quantum Neural Networks to play Doom

Goh Zhen Rong (U2320219A) – zgoh033@e.ntu.edu.sg
Vannes Wijaya (U2320201J) – VWIJAYA002@e.ntu.edu.sg
Borowski Michal (N2601357D) – N2601357D@e.ntu.edu.sg
Jayaraman Vijayaraghavan (U2323763F) – vijayara002@e.ntu.edu.sg

Academic Year 2026/2027 – Semester 1

Deliverable Item	Submission Link / Access URL
Recorded Presentation (15 Mins):	[Insert Google Drive or YouTube URL]
Source Code Repository:	[Insert GitHub repository URL]

Abstract

Games serve as valuable testbeds for developing and evaluating algorithms and machine learning models, offering controlled environments in which their capabilities can be demonstrated and benchmarked. In 2024, Freinberger et al. showed that a hybrid model combining a parametrized quantum circuit with classical encoding and post-processing layers can solve Atari Pong and reach scores comparable to a classical reference on Breakout [1]. Building upon this work, we ask the research question: Can a QNN play more complex games with any quantum advantage? Our agent is a classical CNN that compresses four stacked frames into 12 angles driving a 12-qubit, two-layer data-re-uploading circuit with 74 trainable parameters, whose Pauli-Z expectations give the action logits. We compare it with a classical MLP head and a linear readout on an identical encoder. Two smaller ViZDoom scenarios first confirmed that the circuit can be trained end to end behind a CNN (96–99% and 98% success). On E1M1, reinforcement learning (PPO) alone never found the route in three attempts, two run to 10M decisions. Students that imitate a scripted teacher with access to private game state, using DAgger, exit E1M1 on 70–77% of 100 held-out seeds (circuit: 73% greedy, 70% sampled; classical MLP 72%/77%; linear 77%/72%; teacher 84%). Perturbation tests show these students rely on level-specific cues and the on-screen HUD.

1 Application Problem and Motivation

1.1 Background and Domain Context

Recent research has demonstrated the potential of hybrid quantum-classical reinforcement learning models in game-playing tasks, including Atari games such as Pong and Breakout [1]. Building on these findings, this project investigates the application of Quantum Neural Networks (QNNs) to a more complex and dynamic environment, the first-person shooter Doom through the ViZDoom platform [2], with the first level of the original game, E1M1 (Hangar), as the benchmark.

The target problem falls under reinforcement learning (RL), specifically visual, sequential decision-making under partial observability. E1M1 requires an agent to interpret high-dimensional visual observations, follow a route of about 5,000 map units through rooms, open doors with a USE action, avoid damaging floors, fight six monsters and finally press an exit switch. These challenges are relevant to real-world applications such as autonomous robotics, self-driving vehicles, and intelligent control systems, where agents must continuously process sensory information and adapt their actions to uncertain environments.

This project explores whether QNNs, through their parameterised quantum circuits and quantum feature representations, can offer an alternative approach to learning effective decision-making policies. Parametrised circuits have been used as policies and Q-functions on small RL benchmarks [**chen_variational_2020**], and careful comparison against classical baselines is essential when judging such models [**bowles_better_2024**]. We aim to train a QNN-based agent to complete E1M1 from visual observations while evaluating its learning performance, sample efficiency, and computational requirements against classical baselines. The objective is not to assume a quantum advantage, but to investigate whether QNN-based architectures offer practical benefits or introduce additional limitations in complex visual reinforcement learning tasks.

Before attempting E1M1 we checked feasibility on two smaller ViZDoom scenarios, a maze (My Way Home) and a combat corridor (Deadly Corridor); these pilots are summarised in Section 4.1. The rest of the report concerns E1M1.

1.2 Problem Formulation

E1M1 is modeled as a partially observable Markov decision process (POMDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \Omega, O, \gamma)$ with $A = |\mathcal{A}| = 7$ discrete actions (forward, attack, strafe left, strafe right, turn left, turn right, use). The objective is to learn a policy that maximizes expected cumulative reward, and completes the level, from visual observations. We compare quantum and classical policy architectures under a common training framework.

- **Input:** The environment produces RGB frames of resolution 160×120 with the heads-up display (HUD) visible, which are downsampled to 80×60. Four consecutive frames are stacked to form $o_t \in \mathbb{R}^{12\times60\times80}$.
  For the classical-based models, we employ a shared CNN encoder f_η, consisting of three convolutional layers with progressively increasing channel dimensions. The architecture comprises an 8×8 convolution with stride 4 and 32 output channels, followed by a 4×4 convolution with stride 2 and 64 output channels, and a final 3×3 convolution with stride 1.
  The CNN encoder extracts a 256-dimensional feature representation, followed by a 12-dimensional bottleneck:
  $$\mathbf{z}_t = \pi \tanh(W_b f_\eta(o_t) + \mathbf{b}_b) \in [-\pi, \pi]^{12}. \tag{1}$$
  The bottleneck representation is provided to one of three actor–critic architectures:
  1. Classical MLP: nonlinear actor and critic (hidden width 32).
  2. Linear: a single affine map giving the policy logits and the value.
  3. Quantum-Hybrid: a 12-qubit variational actor–critic with two data-re-uploading layers; the action logits and the value estimate are both Pauli-Z expectation values of the circuit (there is no classical critic).

  Each actor outputs logits over the A discrete actions, defining $\pi(a_t|\mathbf{z}_t)$ through softmax. Position, depth, object labels and the kill counter are read from the engine only to compute the reward, auxiliary training targets and the teacher's labels; they are never inputs to the policy.
  We also attempt to use a fully quantum model to play DOOM. Instead of using a CNN to encode the input, only o_t is used. For each screen, Canny edge density mapped to [-1,1] was used to to separate the enemies in the game. The goal is separated via a green color override so it would be stand out to the agent.
- **Objective / Cost Metric:** The reinforcement learning objective maximizes expected discounted return:
  $$J = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]. \tag{2}$$
  We first optimize all architectures with PPO [3]. The clipped policy loss is
  $$\mathcal{L}_{\text{policy}} = -\mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right], \tag{3}$$
  where
  $$\rho_t = \frac{\pi(a_t|\mathbf{z}_t)}{\pi_{\text{old}}(a_t|\mathbf{z}_t)} \tag{4}$$
  and $\hat{A}_t$ is the advantage estimated using Generalized Advantage Estimation (GAE) [**schulman_high-dimensional_2016**]:
  $$\delta_t = r_t + \gamma(1-d_t)V(\mathbf{z}_{t+1}) - V(\mathbf{z}_t), \tag{5}$$
  $$\hat{A}_t = \delta_t + \gamma\lambda(1-d_t)\hat{A}_{t+1}, \tag{6}$$
  where d_t indicates episode termination. Advantages are normalized within each minibatch.
  The critic (the MLP critic, the linear value readout, or the circuit's value qubit) minimizes the value loss:
  $$\mathcal{L}_{\text{value}} = \frac{1}{2}\mathbb{E}_t \left[(V(\mathbf{z}_t) - \hat{G}_t)^2 \right], \tag{7}$$
  with $\hat{G}_t = \hat{A}_t + V_{\text{old}}(\mathbf{z}_t)$. The complete PPO training objective is
  $$\boxed{\mathcal{L} = \mathcal{L}_{\text{policy}} + c_v\mathcal{L}_{\text{value}} - c_e\mathcal{H}(\pi) + c_a\mathcal{L}_{\text{aux}}} \tag{8}$$
  where $\mathcal{L}_{\text{aux}}$ comprises auxiliary losses used to train the shared visual representation: mean-squared error on per-sector depth, binary cross-entropy on whether a monster is visible in each of eight screen sectors and whether a door or the exit switch is directly ahead, and (in some runs) mean-squared error on the direction and progress along the route. They are used only during training [4, 5].
  Imitation objective. PPO from scratch did not learn E1M1 (Section 4.2), so the policies that clear the level are trained by DAgger imitation [6] of a scripted teacher π_T that reads private game state. With teacher action $a_t^\star = \pi_T(s_t)$ on the states o_t visited by the current student, the loss is
  $$\mathcal{L}_{\text{BC}} = -\mathbb{E}_{o_t\sim\mathcal{D}}\left[w_{a_t^\star} \log \pi(a_t^\star|\mathbf{z}_t) \right] + c_a\mathcal{L}_{\text{aux}}, \qquad w_a \propto f_a^{-0.75}, \tag{9}$$
  where f_a is the frequency of action a in the aggregated dataset $\mathcal{D}$, so that rare actions such as USE are not ignored. After each round the student drives, the teacher labels the visited states, and they are added to $\mathcal{D}$.
- **Reward:** The environment reward is
  $$r_t = -0.01 + 100\,\mathbf{1}[\text{exit}] - 30\,\mathbf{1}[\text{death}] - 0.05\,\Delta\text{HP}_t^- + 2\,\Delta\text{kills}_t + 30\big(\gamma\Phi(s_{t+1}) - \Phi(s_t)\big), \tag{10}$$
  with the potential-based progress term $\Phi(s) = \max(0, 1 - c(s)/c_0)$ and $\Phi = 0$ at episode end. Here $c(s)$ is the walking cost from the player's position to the exit, taken from a path-planner field, since the route winds through rooms and doors and straight-line distance would reward walking into walls; $c_0 = 318.6$ is the cost at the start. The position and the field are used only inside the reward.
- **Scope and Constraints:** Experiments use E1M1 at skill 3 from the real start, with 6 monsters, three doors and a limit of 1,575 decisions per episode (3 minutes of game time; a decision is one policy action held for 4 game tics).

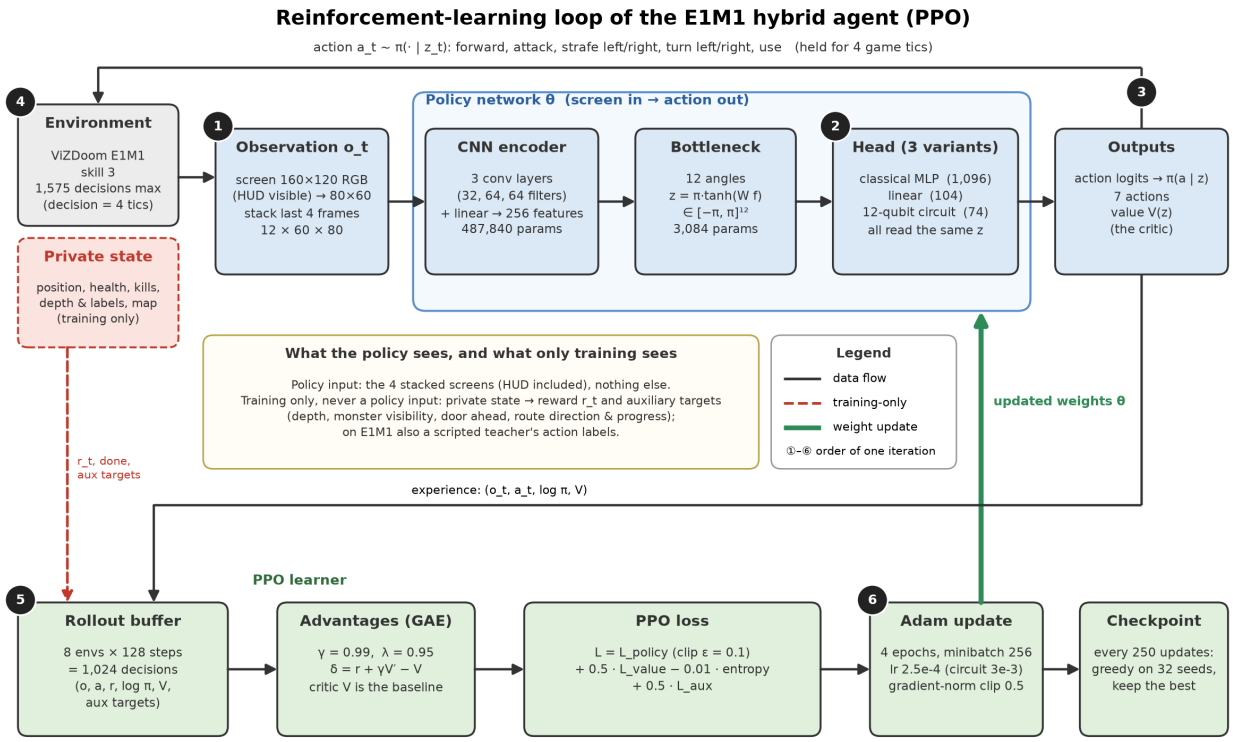


Figure 1: PPO Training

2 Quantum Method and Technical Correctness

2.1 Quantum Algorithm / Model Architecture

We use a variational quantum circuit as the policy and value readout of the agent. It combines angle encoding, data re-uploading [7] and an ansatz in the hardware-efficient style, namely single-qubit rotations with a CNOT ring [8]. The circuit has $n = 12$ qubits and $L = 2$ layers, reads the bottleneck $\mathbf{z} \in [-\pi, \pi]^{12}$ and has 74 trainable parameters.

1. **State Preparation / Feature Map:** Each bottleneck value z_i is encoded as the angle of an R_Y rotation on its own qubit, scaled by a learned factor $s_{\ell,i}$ (initialised to 1), starting from $|0\rangle^{\otimes 12}$. The same $\mathbf{z}$ is re-uploaded in every layer, with separate scales per layer, so the first-layer feature map $U_\Phi(\mathbf{z}) = \bigotimes_i R_Y(s_{1,i} z_i)$ gives the product state $\bigotimes_i \left(\cos\frac{s_{1,i} z_i}{2}|0\rangle + \sin\frac{s_{1,i} z_i}{2}|1\rangle\right)$.
2. **Parameterized Quantum Circuit (Ansatz):** Layer $\ell \in \{1, 2\}$ applies a trainable single-qubit block followed by a ring of CNOT gates,

$$V_\ell(\mathbf{z}) = \bigotimes_{i=0}^{11} R_X(\beta_{\ell,i})\, R_Z(\alpha_{\ell,i})\, R_Y(s_{\ell,i} z_i), \tag{11}$$

$$E = \mathrm{CNOT}_{11\to 0}\, \mathrm{CNOT}_{10\to 11} \cdots \mathrm{CNOT}_{1\to 2}\, \mathrm{CNOT}_{0\to 1}, \tag{12}$$

$$|\psi(\mathbf{z};\boldsymbol{\theta})\rangle = E\, V_2(\mathbf{z})\, E\, V_1(\mathbf{z})\, |0\rangle^{\otimes 12}. \tag{13}$$

   The trainable parameters are $\boldsymbol{\theta} = \{\alpha, \beta, s\}$ (each 2×12, so 48 angles and 24 scales) and two scalar output gains g_π, g_v, giving 74 in total. Angles are initialised as $0.05\,\mathcal{N}(0, 1)$, with $g_\pi = 2$ and $g_v = 5$.
3. **Measurement Observable:** Single-qubit Pauli-Z observables $\hat{M}_k = Z_k$ are measured on the first $A + 1 = 8$ qubits:

$$\ell_k = g_\pi \langle\psi|Z_k|\psi\rangle \quad (k = 0, \ldots, 6), \qquad V = g_v \langle\psi|Z_7|\psi\rangle. \tag{14}$$

   The ℓ_k are the softmax logits and V is the value estimate; qubits 8–11 are not measured but influence the outputs through the entanglement. All observables commute, so one circuit execution in the computational basis provides all of them. There is no learned classical layer after the circuit; the gains only set the output scale. Under imitation only the logits are trained, so the value gain stays at its initial value.

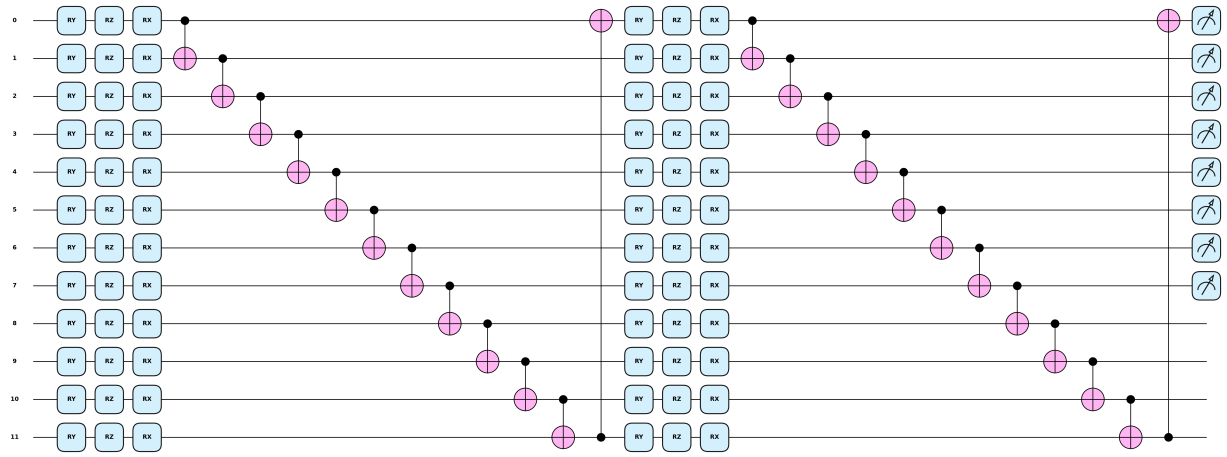


Figure 2: QML Doom Playing Model

3 Implementation and Experimental Design

3.1 Software Environment

The implementation is developed in Python 3.12 using PyTorch 2.14.1 for classical neural network components, PennyLane 0.45.1 for quantum circuit simulation, and ViZDoom 1.3.2 with Gymnasium 1.4.0 for the environment. PennyLane integrates with PyTorch to enable end-to-end training of the hybrid quantum-classical neural network. Quantum circuits are simulated using PennyLane's `default.qubit` simulator with the PyTorch interface and `backprop` differentiation, assuming ideal, noiseless quantum operations. Training uses a Slurm GPU cluster (NVIDIA V100), with one ViZDoom process per environment (8 in parallel) running on the CPU cores.

3.2 The E1M1 Environment, Teacher and Training Protocol

Environment. E1M1 is read from the shareware `doom1.wad`. A route planner (Dijkstra on a 16-unit grid) gives a 22-waypoint route of about 4,972 units with doors at waypoints 5, 17 and 20, and a reverse Dijkstra from the exit gives the walking-cost field behind the progress reward. A door that is already opening closes again if USE is pressed, so the agent must press USE once and then walk. For training only, an episode can start at a route waypoint (never a door waypoint) facing the route within ±45°, which gives practice on every segment. For evaluation, an episode always starts at the real start.

Scripted teacher. The teacher reads private state: position and heading, the walking-cost field (for the route direction) and door and switch geometry. Monster positions come from the labels buffer, as a player would see them. Each decision it (1) fires if a monster at least 12 pixels tall is near the screen centre, (2) presses USE once when a door or the exit switch is within 40 units ahead, with a 25-decision cooldown, or (3) turns toward the route and walks, turning in place if it is stuck. The teacher exits on 84 of 100 held-out seeds at skill 3 (29 of 30 at skill 1).

DAgger protocol. Round 0 trains on 200K teacher-driven decisions. Each of 8 further rounds lets the student drive for 60K decisions, labels every visited state with the teacher's action, and retrains on all data collected. Episodes start at the real start (25%) or a random non-door waypoint, and are cut at 600 decisions.

3.3 Experimental Design and Baselines

Each experiment (configuration in Table 1) compares three policy heads on the same encoder and bottleneck: the 12-qubit circuit (74 parameters), a classical MLP actor–critic (1,096 parameters) and a linear readout (104 parameters). Trivial baselines are random actions and always walking forward. A scripted teacher that reads private state provides an upper reference, and behaviour cloning on teacher demonstrations alone and PPO without any teacher are the ablations.

We use completion as the main metric: the fraction of episodes in which the agent presses the exit switch while alive, on 100 held-out spawn seeds (20000–20099) not used in training or selection, with greedy action selection and with actions sampled from the policy. Best checkpoints are chosen on 32 separate seeds (10000–10031). As secondary metrics we report deaths, kills and route progress, together with wall-clock time.

3.4 Reproducibility

The codebase layout, environment setup and sample commands for reproducing every experiment are given in the repository's `README.md`.

Table 1: Experimental configuration. All heads use the same encoder, optimiser settings and seeds, except where noted.

Configuration Parameter	Quantum Setup	Classical Baselines
Policy head	12-qubit circuit, 2 layers, 74 parameters	MLP 12→32→ 7 (+ critic): 1,096; linear: 104
Shared encoder	3 conv layers (32, 64, 64 filters) + linear 1536→256 + bottleneck 256→12: 490,924 parameters	
Framework / Backend	PennyLane `default.qubit`, exact statevector	PyTorch
Imitation (final method)	DAgger, 200K teacher states then 8 rounds of 60K student states; Adam, encoder lr 3×10^{-4} (circuit lr 3×10^{-3}); cross-entropy with class weights $f_a^{-0.75}$ and auxiliary losses	
PPO (attempts)	8 envs × 128 steps per update, 4 epochs, minibatch 256, $\gamma = 0.99$, $\lambda = 0.95$, clip 0.1, $c_e = 0.01$, $c_a = 0.5$; Adam, lr 2.5×10^{-4} (circuit lr 3×10^{-3})	
Measurement sampling	Statevector (exact), no shots	Deterministic network
Evaluation metrics	Exits on 100 held-out seeds (greedy and sampled), deaths, kills, route progress, wall-clock time	

4 Results and Critical Analysis

4.1 Feasibility Pilots on Two Smaller Scenarios

To check that a circuit head can be trained end to end behind a CNN at all, we first ran the same encoder, heads and PPO on My Way Home (a maze; 3 actions; reach a vest) and Deadly Corridor (6 actions; fight six enemies and reach a vest), with three training seeds per head (one for the corridor circuit). Table 2 shows that the circuit solves both, matching the classical MLP and the linear readout, at about ten times the wall-clock cost; a linear readout suffices. The pilots do not test E1M1's difficulties (a long route, doors, nukage, mixed combat), so they establish feasibility of the circuit head, not of E1M1.

Table 2: Feasibility pilots (PPO), greedy exits out of 100 held-out seeds, listed as seed 0/1/2. Random actions score 10 (maze) and 0 (corridor).

Scenario	Head (params)	Greedy exits; decisions to threshold[†]	Wall time
My Way Home	Classical MLP (964)	99/100/100; 1.28M/0.77M/1.28M	~31 min
	Linear (52)	99/98/96; 0.77M/0.77M/0.77M	~31 min
	12-qubit circuit (74)	96/99/97; 1.79M/2.56M/1.79M	5 h 25 min[‡]
Deadly Corridor	Classical MLP (1,063)	99/99/99; 2.56M/1.79M/2.82M	~31 min
	Linear (91)	98/100/100; >3M/2.82M/1.79M	~30 min
	12-qubit circuit (74, 1 seed)	98; 4.35M	~11 h

[†]Selection score ≥ 28/32 (maze) or 32/32 (corridor). [‡]Stopped by the 19,500 s budget just short of 3M decisions.

4.2 E1M1: Reinforcement Learning Alone Did Not Work

Table 3 summarises the PPO attempts on E1M1 with the classical head (up to 10M decisions each). None found the route: the best real-start evaluation had route progress 0.15–0.25, against 0.21 for random actions and 0.11 for always walking forward. The final policies were degenerate (mostly strafing, or pressing USE on 93% of decisions). Videos of the sampled policies show undirected behaviour: turning in place and firing until ammunition runs out, or turning away from a zombieman while health falls.

Table 3: PPO attempts on E1M1 (classical head, real-start evaluation).

Attempt	What changed	Outcome
1	Reverse-start curriculum (start near the exit, move back as success rises), corridor-encoder warm start, 10M decisions	0/100 exits. Stuck at the stage starting at waypoint 8 from 2.5M decisions on; greedy policy mostly strafed
2	Uniform random route starts, no stages	Progress gain per episode fell from 0.15 to 0.07 by 3M decisions; 0/32 (cancelled)
3	Plus route-direction and route-progress auxiliary targets, 10M decisions	Encoder learned the targets (loss 0.19 to 0.04) but 0/100 exits; greedy policy collapsed to USE
4	PPO fine-tuning of the imitation student (naive; frozen encoder with a KL anchor; larger kill reward; critic on 256 features)	Naive: collapsed to 0/64 within 100 updates. Others stable but flat; best checkpoint: 255 against 238 of 300 greedy exits (about 1.8 standard errors), kills not higher

4.3 E1M1: Imitation of a Scripted Teacher

Table 4 gives the main result. A screen-only network clears E1M1 on 48–58% of unseen seeds from teacher demonstrations alone (63–78% sampled), and DAgger raises greedy play to 72–77% at the same 680K-state budget, a gap of about two standard errors for the classical head while sampled play is unchanged (77 against 78). So DAgger's benefit is modest and, with one seed, not firmly established, whereas having a teacher is decisive. A first version of the teacher pressed USE from up to 72 units away and then walked on while the door opened, so a closed door in view was labelled USE only about one time in four; its greedy student never pressed USE and died at the doors at waypoints 17 and 20 (37 exits greedy against 83 sampled). Pressing USE only within 40 units and weighting rare actions fixed this (72 greedy). The remaining failures are mostly at the waypoint-17 door.

Table 4: E1M1, skill 3, real start, 100 held-out seeds (one training seed each).

Method	Head params	Exits, greedy	Exits, sampled	Died, greedy
PPO from scratch (best of 3 attempts)	1,096	0	0	0%
Behaviour cloning only, 200K teacher states	1,096	48	63	52%
Behaviour cloning only, 680K teacher states	1,096	58	78	42%
DAgger, classical MLP head	1,096	72	77	28%
DAgger, linear head	104	77	72	23%
DAgger, 12-qubit circuit head	74	73	70	27%
Scripted teacher (reads private state)	–	84	84	16%
Random actions	–	0	–	46%

4.4 Analysis of Convergence, Scalability, and Circuit Depth

- **Convergence Behavior:** Under imitation all heads see the same data schedule, so the difference is in fit and cost. The circuit fits the teacher's labels less tightly (training accuracy 87% against 91–92% for the classical head, cross-entropy 0.31 against 0.24) and took 1 h 53 min against about 25 min. Selection exits on 32 seeds fluctuate by several episodes between rounds (classical 18–29, linear 22–29, circuit 17–26), so a single seed cannot rank rounds or heads. We did not diagnose barren plateaus.
- **Effect of Circuit Depth:** We used $L = 2$ throughout and did not sweep the depth, so we have no depth–quality–runtime curve for E1M1. The pilots did show the circuit slower than classical heads in decisions as well as seconds: 1.79–2.56M against 0.77–1.28M decisions on the maze and 4.35M against 1.79–2.82M on the corridor, though one circuit seed cannot separate that from seed variation (Table 2).
- **Does the circuit matter?** On E1M1 (Table 4) and on both pilots the linear readout matches the circuit and the classical MLP; the CNN encoder does the perception and any small head can read it out. The circuit's 74 parameters are 0.015% of the model.
- **Do the students memorise the level?** E1M1 is one fixed level, so held-out seeds change only monster behaviour. Perturbation tests without retraining (Table 5) show the students depend on level-specific cues: started at a random reachable position and heading, exits fall from about 75–82 to 25–30 while the teacher gets 92; blanking the HUD (health, ammo) takes every student to 0 exits, although the classical student tolerates noise and brightness changes. The students also fail on the pilot scenarios (6–10 greedy exits on the maze against 10 for random actions, and 0–1 on the corridor), and the pilot-trained models make no progress on E1M1. So the result reflects learning one level, not general skills.

Table 5: Memorisation tests on the E1M1 students: exits out of 100 (greedy), no retraining, episodes cut at 800 decisions.

Condition	Classical	Linear	Circuit
Unmodified screens, real start (teacher: 82)	76	82	73
HUD masked (bottom 10 of 60 rows set to 0)	0	0	0
Four stacked frames replaced by the newest	66	0	47
Brightness $\times 0.6$ / $\times 1.4$	68 / 75	79 / 72	0 / 58
Gaussian noise (std 25 of 255)	77	73	65
Random off-route start (teacher: 92)	25	30	30

4.5 Critical Limitations and NISQ Feasibility

- **Where the learning happens.** The classical parts (encoder, bottleneck and auxiliary head) hold 99.985% of the parameters, and a linear readout does as well as the circuit. Our results show a circuit can be trained end-to-end behind a CNN to clear E1M1, and they show no benefit of doing so.
- **A privileged teacher was needed.** PPO scored 0 exits in three attempts. The students that clear E1M1 learn from a hand-written teacher that reads position, route and door geometry, so they measure how well a screen-only network can copy an expert, not whether reward alone finds the behaviour.
- **Memorisation.** The students recognise places along one route, helped by the HUD, and are lost away from it; they do not transfer to other tasks.
- **Cost.** Simulation makes the circuit about ten times slower per update, and statevector memory doubles with every added qubit.
- **Statistical strength.** Every E1M1 number uses one training seed and 100 test seeds; the differences between the heads (70–77 exits) are within noise.
- **NISQ feasibility.** The circuit has 24 CNOTs in a serial chain (depth about 30) with a wrap-around gate that needs ring connectivity or SWAPs. At typical two-qubit error rates of 0.5–1%, the CNOT errors alone would leave roughly 79–89% of the ideal state fidelity (0.99^{24} to 0.995^{24}), before readout error and decoherence; this is a rough estimate, not a hardware measurement. Training on hardware would need 144 shifted circuits per sample per step, so the practical use is inference only, with one circuit execution per decision and enough shots to resolve the logits.

5 Complementary Experiment: Hybrid QNN with Explicit Navigation

5.1 Architecture and Training

Alongside the screen-only E1M1 experiment, we developed a six-qubit hybrid quantum-classical policy for several full-level DOOM scenarios. Unlike the main experiment, this approach incorporates privileged game-state features and explicit navigation. Its higher completion rates therefore do not constitute a direct comparison with the screen-only model.

The agent processes a 64×96 greyscale frame together with structured features, including enemy bearing and distance, depth rays, inventory information and direction to the next planner waypoint. A CNN and feature encoder produce a 48-dimensional representation, which is projected to six angles in $[-\pi, \pi]$. These are encoded through R_Y gates on six qubits, followed by two layers of trainable R_Y and R_Z rotations with CNOT-ring entanglement [8]. Pauli-Z expectation values contribute to the action logits alongside a classical residual pathway. A learned gain g_q controls the quantum contribution, allowing it to be disabled at evaluation by setting $g_q = 0$.

The circuit is evaluated using a custom complex64 PyTorch statevector simulator. Cross-checks against PennyLane found a maximum expectation difference of 6.6×10^{-7} and identical action choices across 2,000 evaluated states; the custom implementation was approximately 3.6× faster in the reported benchmark. Policies are trained primarily through behaviour cloning from a scripted teacher handling navigation, doors and combat. Selected configurations additionally use DAgger [6]. The geometry-aware planner provides route guidance during gameplay, reducing the navigation burden on the learned policy.

5.2 Full-Level Evaluation

Table 6 summarises the reported completion rates across four levels. The system also achieved 93%, 77% and 23% success on Deadly Corridor at difficulties 1, 3 and 5, respectively (30 episodes each).

Table 6: Complementary hybrid policy results. Evaluation protocols differ between scenarios; these are not zero-shot transfer results.

Scenario	Exits	Success
DOOM E1M1	49/50	98%
DOOM E1M2	153/170	90%
DOOM E1M3	18/40	45%
Freedoom MAP01	19/20	95%

The MAP01 result illustrates the importance of classical navigation. With the original planner, the quantum student completed only 1/20 episodes. After improving the planner, the same trained checkpoint completed 19/20 without retraining; a classical student achieved 20/20 under the improved setup. Thus, the substantial improvement was obtained through navigation changes rather than additional quantum learning.

5.3 Quantum Ablations and Limitations

To assess the circuit's contribution, we evaluated the residual policy with its quantum branch disabled and compared it with classical alternatives. Table 7 summarises the E1M2 ablations on the same 70 evaluation seeds.

Table 7: E1M2 quantum ablations (70 evaluation episodes).

Configuration	Exits
Residual hybrid QNN	66/70
Same checkpoint, quantum disabled ($g_q = 0$)	64/70
Classical model, retrained	62/70
Quantum circuit without classical residual	50/70

Disabling the quantum branch reduces E1M2 completions by only two episodes, while a separately trained classical model performs similarly. Removing the classical residual has a larger effect, suggesting that the classical pathway accounts for much of the successful control. On E1M3, however, disabling the quantum branch reduced completion from 18/40 to 0/40, indicating that this checkpoint depended more strongly on its quantum outputs. Without a retrained classical E1M3 baseline, this does not establish a quantum advantage.

These results demonstrate that a small quantum circuit can be integrated into a capable hybrid agent, but they do not establish a measurable benefit over classical alternatives. The dependence on privileged features and explicit planning, limited independent training seeds, and ideal statevector simulation constrain the conclusions. Together with the main experiment, the findings suggest that representation quality, imitation learning and navigation are more important to the observed performance than the choice of quantum or classical policy head.

6 Conclusion and Future Directions

This study investigated the feasibility of QNNs in complex game-playing environments through two complementary hybrid quantum-classical approaches. Using ViZDoom, the effectiveness of variational quantum circuits in supporting navigation, combat, and decision-making was evaluated against classical alternatives.

The first approach employed a CNN encoder with a 12-qubit data-re-uploading circuit to learn directly from visual observations. Initial reinforcement learning experiments using Proximal Policy Optimization (PPO) with a classical policy head failed to achieve successful level completion in E1M1. However, imitation learning through DAgger enabled the quantum agent to achieve a 73% completion rate under greedy evaluation. Classical MLP and linear alternatives achieved comparable results of 72% and 77%, respectively, with substantially lower computational costs. Furthermore, the trained policies exhibited limited generalisation beyond the original environment, indicating a reliance on level-specific visual features.

The second approach incorporated a six-qubit variational quantum circuit alongside explicit navigation and structured environmental information. This architecture achieved completion rates of 98%, 90%, and 45% across E1M1, E1M2, and E1M3, respectively. Ablation studies highlighted the importance of classical components and navigation mechanisms, although the quantum pathway contributed to the behaviour of certain trained policies. Nevertheless, these findings did not establish a statistically reliable quantum advantage, particularly given the reliance on privileged simulator information and the limited number of independent training runs.

Collectively, these findings demonstrate the feasibility of integrating QNNs into agents capable of completing complex gameplay tasks. However, the observed performance was largely dependent on classical feature extraction, imitation learning, and explicit navigation rather than a demonstrated advantage of quantum computation.

Future research should prioritise improving experimental reliability and generalisation. Evaluations across multiple independent training seeds, consistent classical baselines, and additional ablation studies would provide stronger evidence regarding the contribution of quantum components. Furthermore, incorporating random off-route starting positions, HUD dropout, and training across multiple environments could reduce reliance on memorised visual features and improve adaptability to unseen scenarios.

Further investigation into quantum circuit architectures, including variations in circuit depth ($L = 1, 2, 3, 4$), entanglement structures, and data-re-uploading strategies, could clarify the relationship between expressivity, computational cost, and learning performance. Recurrent architectures, such as classical Gated Recurrent Units (GRUs) and Quantum Long Short-Term Memory (QLSTM) models [9], could also be explored to address limitations in sequential decision-making under partial observability. Finally, evaluating finite-shot measurements, hardware noise, and noise-mitigation techniques such as zero-noise extrapolation would provide a more realistic assessment of scalability and feasibility on noisy intermediate-scale quantum (NISQ) devices.

Overall, this study establishes the feasibility of hybrid quantum-classical architectures for complex game-playing tasks while highlighting the need for further controlled experimentation to determine whether quantum components offer meaningful advantages over conventional machine learning approaches.

7 Group Member Contribution Breakdown

As required by the group project guidelines, detail each member's specific technical and editorial contributions:

Table 8: Group Member Contributions and Task Allocation

Member Name	Key Responsibilities and Deliverables	Presentation Video Role
Goh Zhen Rong	[TODO]	[TODO]
Vannes Wijaya	[TODO]	[TODO]
Borowski Michal	[TODO]	[TODO]
Jayaraman Vijayaraghavan	[TODO]	[TODO]

References

[1] Dominik Freinberger, Julian Lemmel, Radu Grosu, and Sofiene Jerbi. *A quantum-classical reinforcement learning model to play Atari games*. 2024. arXiv: 2412.08725.

[2] Michał Kempka, Marek Wydmuch, Grzegorz Runc, Jakub Toczek, and Wojciech Jaśkowski. "ViZDoom: A Doom-based AI research platform for visual reinforcement learning". In: *IEEE Conference on Computational Intelligence and Games (CIG)*. 2016, pp. 341–348.

[3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347.

[4] Piotr Mirowski et al. "Learning to navigate in complex environments". In: *International Conference on Learning Representations (ICLR)*. 2017.

[5] Guillaume Lample and Devendra Singh Chaplot. "Playing FPS games with deep reinforcement learning". In: *AAAI Conference on Artificial Intelligence*. 2017.

[6] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. “A reduction of imitation learning and structured prediction to no-regret online learning”. In: *International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 15. PMLR. 2011, pp. 627–635.

[7] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. “Data re-uploading for a universal quantum classifier”. In: *Quantum* 4 (2020), p. 226.

[8] Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M. Chow, and Jay M. Gambetta. “Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets”. In: *Nature* 549 (2017), pp. 242–246.

[9] Samuel Yen-Chi Chen, Shinjae Yoo, and Yao-Lung L. Fang. *Quantum Long Short-Term Memory*. 2020. arXiv: 2009.01783.